# Day-5

| Day | Date | Content Delivery Date | Topics | Comments |
|---|---|---|---|---|
| Day 1 : | 08 November 2025 | Nov 4th | > Creating new device and adding Lunch option in AAOS<br>> Benefits of AIDL as Interface Description in Android-14<br>> Transitioning from HIDL to AIDL<br><br>> AAOS Media Player<br>- Architecture<br>- Media Source detection and streaming<br>- Play/puase, next/prev, fwd/rewind event flow and handling<br>> Audio Manager:<br>- Source switching<br>- Audio attributes and priority management<br>- Early Audio<br>- IVI and cluster sounds | All the media player needs to be develeoped on android, also include, Source Switiching, Audio Attributes, early audio. |
| Day2: | 15 November 2025 | Nov 11th | > Using SOME/IP in integrated cockpit enviornment<br>- Service discovery<br>- Comm between integrated cockpit ECUs<br>> Using VOIP in integrated cockpit enviornment<br><br>> IPC Mechanism between AAOS Layers<br>> IPC between IVI and Cluster using Telechip SoC<br>> Multidisplay mgmt , android and cluster (trainer will check and revert)<br><br>> Integrating 3rd party apps like Spotify, YouTube etc. with AAOS | As Spotify integration can happen either with GAS or by having Spotify Automotive SDK from Spotify.<br>Hence, we will go through the steps of integration only (with and without GAS). |
| Day3: | 22 November 2025 | Nov 18th | Security Measures in Android Automotive<br>- Secure boot, encryption methods, and secure communication.<br>- SE Linux security policies<br>- Secure signing process<br>- Multiuser Management<br>- Setting up user authentication ( PIN-based). | |
| Day4: | 29 November 2025 | Nov 25th | - Android Bootup optimization<br>- V2X<br>- GAS<br>- Future trends | All theoritical discussions |
| Day 5 | 6th December 2025 | | Recap session | |
| | HW/SW setup | | Platform Setup<br>The Hardware Platform: Telechip D3 TCX805x EVB<br>IVI: Android Automotive OS-14<br>Cluster: Linux (Yocto release provided by Telechip) | |

# Agenda -

- Morning Slots
  - HIDL HAL Demo
  - Android Bootup optimization
  - V2X
  - GAS
  - Future trends

- Afternoon Slots
  - IPC between IVI and Cluster using Telechip SoC
  - Audio flow
  - VoIP

# Hardware Abstraction Layers (HIDL Interface)

## What is the HAL for?

- The Hardware Abstraction Layer connects system services in the Framework to the hardware
- OEMs can tune the HAL to work with their specific hardware
- HALs are part of the BSP, located on the vendor partition
- Most HALs are implemented as daemons (native services)
- Uses Binder message passing
- Beginning in O/8, HAL interfaces defined in **HIDL** (Hardware Interface Definition Language)
- From R/11, HAL interfaces transition from HIDL to **Stable AIDL** (Android Interface Definition Language)

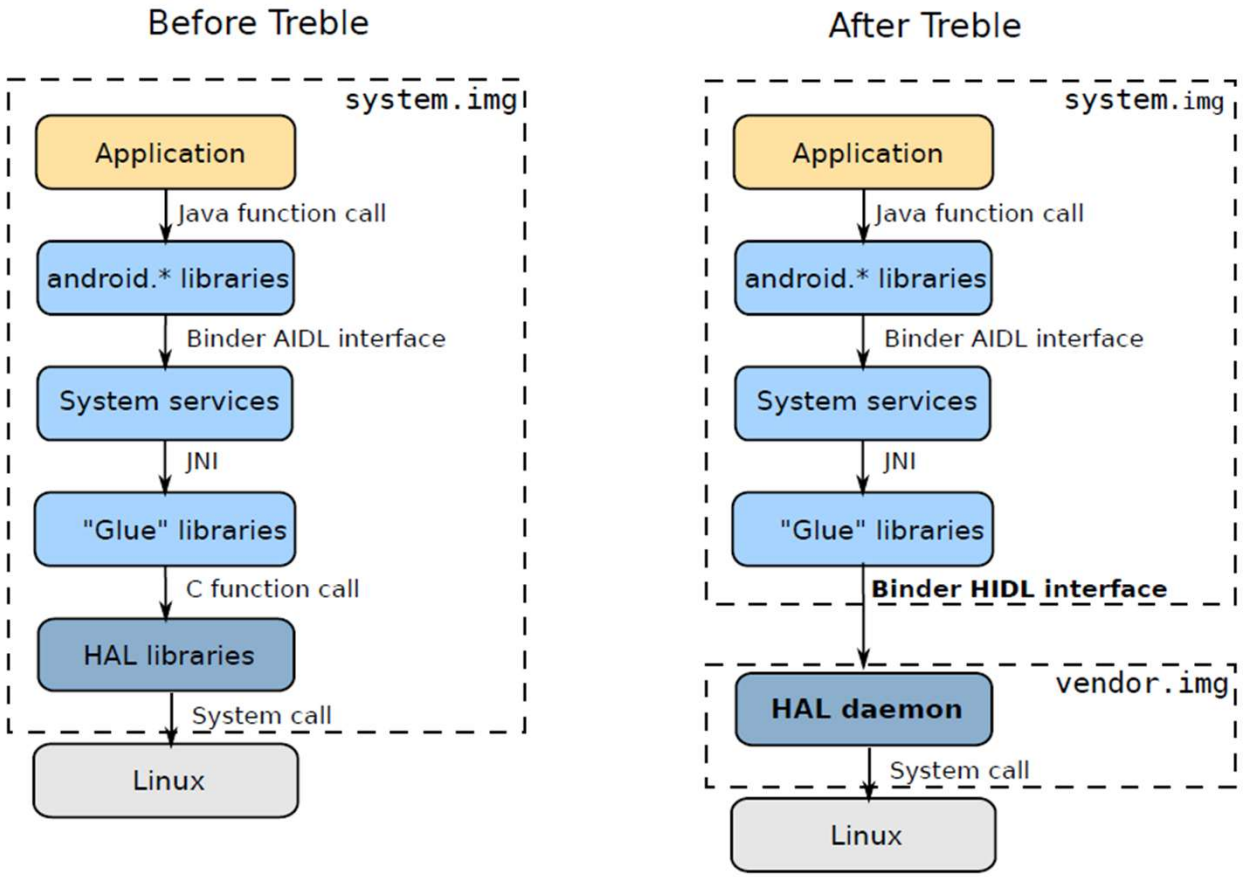
# Project Treble: objectives

- Prior to O/8, everything (framework, HAL, pre-installed apps) was combined into a single image file, system.img
- In O/8, as part of **Project Treble**, Android was split into two parts: **system** and **vendor**
- In recent versions of AOSP -
  - system: contains framework, framework extensions and pre-installed apps deployed across system.img, system_ext.img and product.img
  - vendor: contains the board support package and device additions in vendor.img and odm.img
- **System and vendor parts can be updated independently**
  - for example, you can update the system part to U/14, leaving the vendor part on T/13
  - makes it easier for OEMs to update a product with a new version of AOSP

# Project Treble: objectives

- The vendor part consists of the HAL and supporting configuration files and libraries
- The HAL API is **strongly versioned** so that old and new HALs can coexist
- The interface uses **message passing** (binder) rather than procedure calls; each HAL is implemented as a daemon (*)
- The HAL versions are documented in the Vendor Interface, **VINTF**
- Dependencies between the vendor and system parts are controlled by linker namespaces that are part of the **VNDK**
- (*) there are exceptions – Its not applicable for passthrough HALs

# Project Treble:



# HALs and applications

- Generally, it is not possible for an application to call a HAL daemon directly
- Prevented by SEPolicy
- Necessary because HIDL does not pass UID/PID to the daemon, so it cannot do any authentication
- (also, Android permissions model does not extend down as far as HAL)

# List of mandatory device HALs

These HALs are marked with `optional="false"` in `hardware/interfaces/compatibility_matrices/compatibility_matrix.5.xml`

| Interface | Version |
|---|---|
| android.hardware.audio | 6.0 |
| android.hardware.audio.effect | 6.0 |
| android.hardware.gatekeeper | 1.0 |
| android.hardware.graphics.allocator | 2.0,2.0,4.0 |
| android.hardware.graphics.composer | 2.1-4 |
| android.hardware.graphics.mapper | 2.1,3.0,4.0 |
| android.hardware.health | 2.1 |
| android.hardware.keymaster | 3.0,4.0-1 |
| android.hardware.power | |

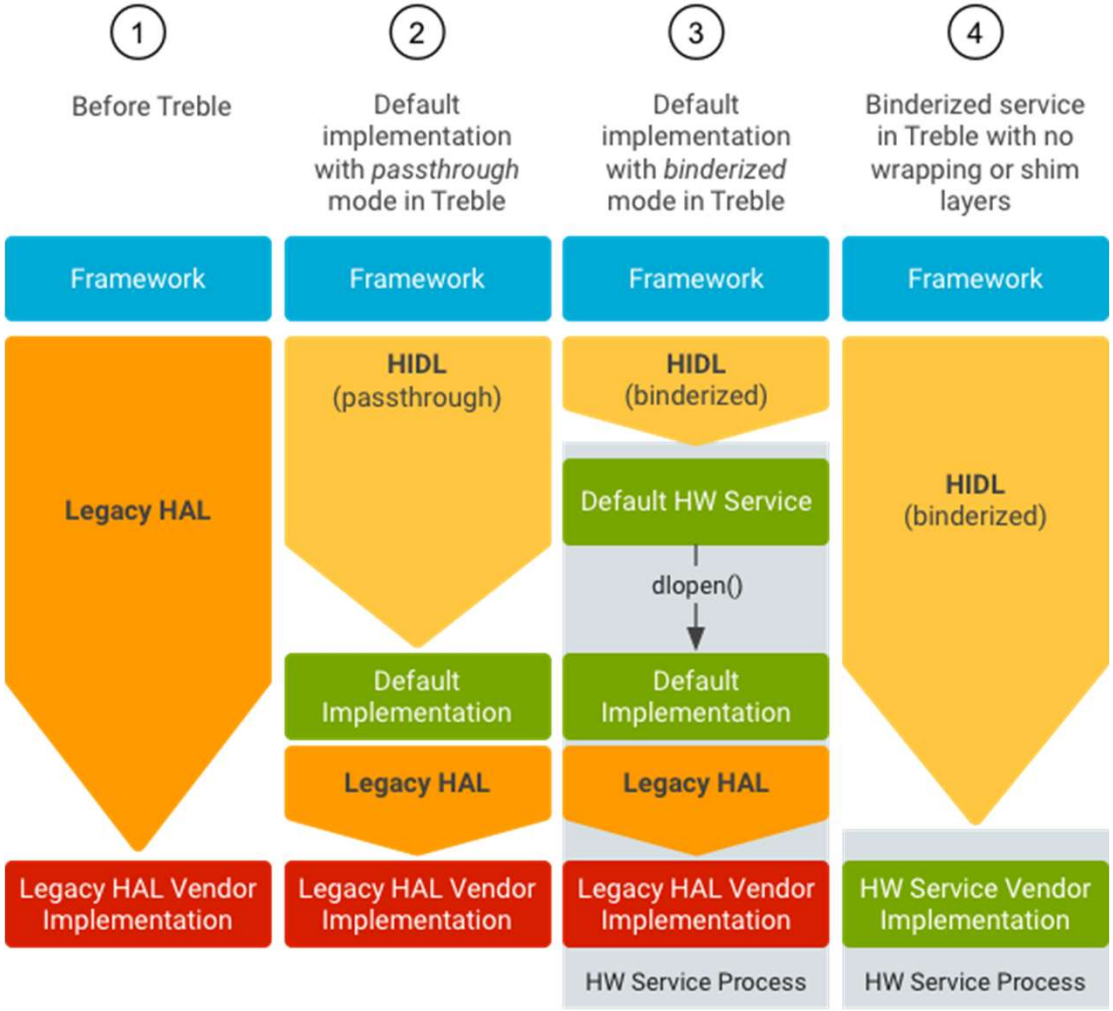
# List of optional device HALs

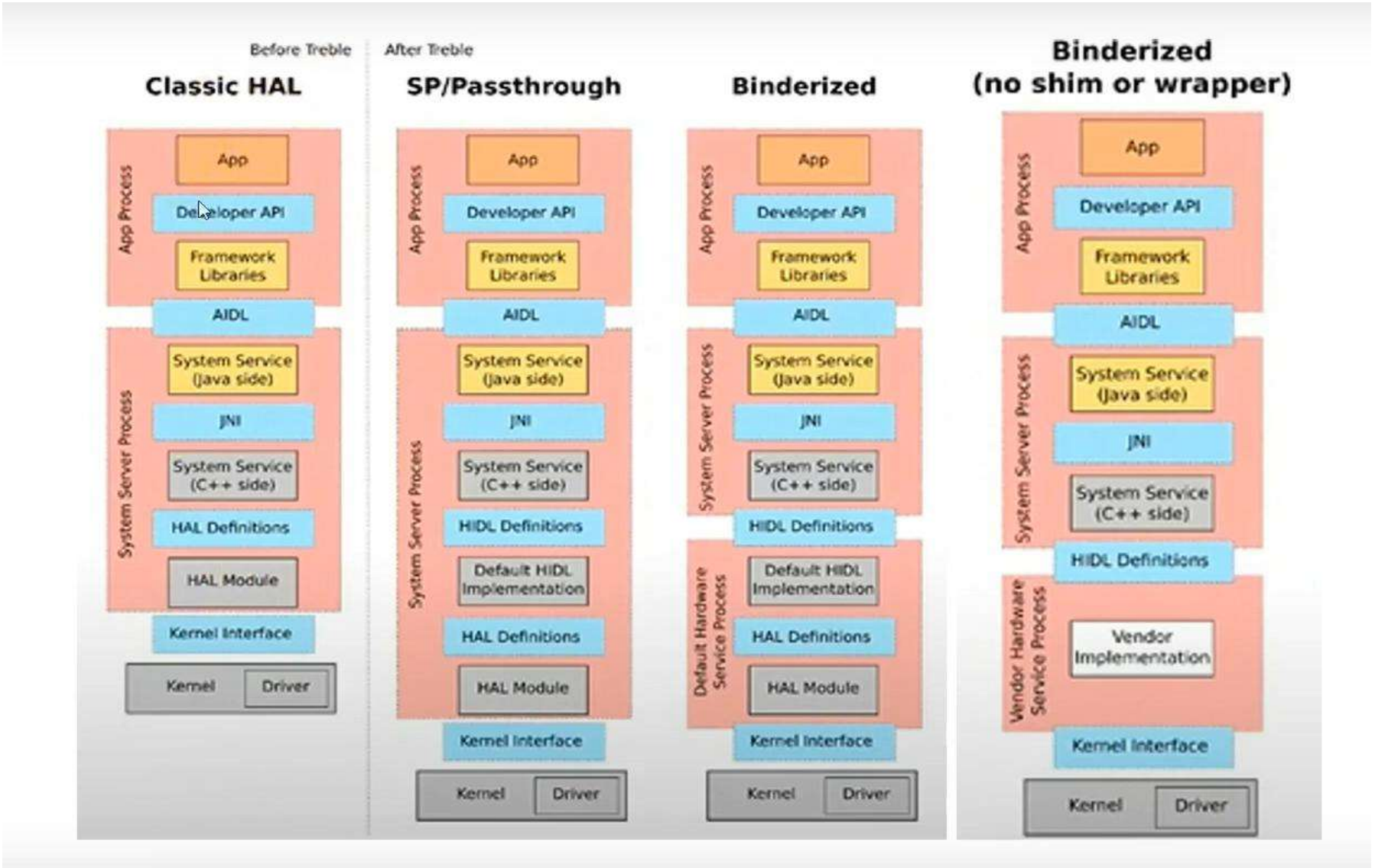
| Interface | Version |
|---|---|
| android.hardware.vibrator | 1.0-3 |
| android.hardware.vr | 1.0 |
| android.hardware.weaver | 1.0 |
| android.hardware.wifi | 1.0-3 |
| android.hardware.wifi.hostapd | 1.0-1 |
| android.hardware.wifi.supplicant | 1.0-2 |

| Interface | Version |
|---|---|
| android.hardware.atrace | 1.0 |
| android.hardware.authsecret | 1.0 |
| android.hardware.biometrics.face | 1.0 |
| android.hardware.biometrics.fingerprint | 2.1 |
| android.hardware.bluetooth | 1.0 |
| android.hardware.bluetooth.audio | 2.0 |
| android.hardware.boot | 1.0 |
| android.hardware.broadcastradio | 2.0 |
| android.hardware.camera.provider | 2.4-5 |
| android.hardware.cas | 1.0 |
| android.hardware.configstore | 1.1 |
| android.hardware.confirmationui | 1.0 |
| android.hardware.contexthub | 1.0 |

| Interface | Version |
|---|---|
| android.hardware.drm | 1.0-2 |
| android.hardware.dumpstate | 1.0 |
| android.hardware.gnss | 2.0 |
| android.hardware.health.storage | 1.0 |
| android.hardware.input.classifier | 1.0 |
| android.hardware.ir | 1.0 |
| android.hardware.light | 2.0 |
| android.hardware.media.c2 | 1.0 |
| android.hardware.media.omx | 1.0 |
| android.hardware.memtrack | 1.0 |
| android.hardware.neuralnetworks | 1.0-2 |
| android.hardware.nfc | 1.2 |
| android.hardware.oemlock | 1.0 |

| Interface | Version |
|---|---|
| android.hardware.power | 1.0-3 |
| android.hardware.power.stats | 1.2 |
| android.hardware.radio | 1.0-2 |
| android.hardware.radio.config | 1.1 |
| android.hardware.renderscript | 1.0 |
| android.hardware.secure_element | 1.0 |
| android.hardware.sensors | 2.0 |
| android.hardware.soundtrigger | 2.0-2 |
| android.hardware.tetheroffload.config | 1.0 |
| android.hardware.tetheroffload.control | 1.0 |
| android.hardware.thermal | 2.0 |
| android.hardware.usb | 1.0-2 |
| android.hardware.usb.gadget | 1.0 |


1
Before Treble
2
Default implementation with *passthrough* mode in Treble
3
Default implementation with *binderized* mode in Treble
4
Binderized service in Treble with no wrapping or shim layers
Framework
Framework
Framework
Framework
Legacy HAL
HIDL (passthrough)
HIDL (binderized)
HIDL (binderized)
Default HW Service
dlopen()
Default Implementation
Default Implementation
Legacy HAL
Legacy HAL
Legacy HAL Vendor Implementation
Legacy HAL Vendor Implementation
Legacy HAL Vendor Implementation
HW Service Vendor Implementation
HW Service Process
HW Service Process


Before Treble
After Treble
Classic HAL
SP/Passthrough
Binderized
Binderized
(no shim or wrapper)
App Process
App
Developer API
Framework Libraries
AIDL
System Server Process
System Service (Java side)
JNI
System Service (C++ side)
HAL Definitions
HAL Module
Kernel Interface
Kernel
Driver
HIDL Definitions
Default HIDL Implementation
Default Hardware Service Process
Vendor Hardware Service Process
Vendor Implementation

# Camera HAL

# Camera HAL mapping

**1) Interface (AIDL)**

- frameworks/av/camera/aidl/

**2) Framework services (system side)**

- **cameraserver** and camera service logic (permission checks, UID policy, app ops, etc.)
  - frameworks/av/services/camera/

- native camera framework libs
  - frameworks/av/camera/

**3) Default / reference HAL implementation**

- Common places:
  - hardware/interfaces/camera
  - hardware/google/ (Pixel-specific/reference components; varies by release)

- In real products, the actual camera HAL is almost always **vendor-specific**
  - vendor/<soc_vendor>/...
  - device/<oem>/<device>/...

- NOTE -> In Telechip Camera HAL implementation is done in -> hardware/telechip/camera folder....

# Telechip Camera HAL architecture (binderized service vs legacy)

- **Check camera provider service**

```
car_tcc8050_arm64:/ # ps -A | grep -i camera
cameraserver  278    1  18240   7404 binder_ioctl_write_read 0 S android.hardware.camera.provider@2.5-service
cameraserver  320    1  36072  14192 binder_ioctl_write_read 0 S cameraserver
```

- **Check HIDL HAL is registered**

```
car_tcc8050_arm64:/ # lshal | grep -i camera
FM    Y android.frameworks.cameraservice.service@2.0::ICameraService/default       0/2      320   183
DM,FC Y android.hardware.camera.provider@2.4::ICameraProvider/legacy/0            0/2      278   320 183
DM,FC Y android.hardware.camera.provider@2.5::ICameraProvider/legacy/0            0/2      278   320 183
```

- **Check init rc**

```
car_tcc8050_arm64:/ # ls /vendor/etc/init | grep -i camera
android.hardware.camera.provider@2.5-service.rc

car_tcc8050_arm64:/ # cat /vendor/etc/init/camera 2>/dev/null
service vendor.camera-provider-2-5 /vendor/bin/hw/android.hardware.camera.provider@2.5-service
  interface android.hardware.camera.provider@2.5::ICameraProvider legacy/0
  interface android.hardware.camera.provider@2.4::ICameraProvider legacy/0
  class hal
  user cameraserver
  group audio camera input drmrpc
  ioprio rt 4
  capabilities SYS_NICE
  writepid /dev/cpuset/camera-daemon/tasks /dev/stune/top-app/tasks
```

# Call flow from Camera / RVC app to HAL (Android 10)

- **TCRearViewcamera App side (RVC / camera app) (It's .apk is in vendor/telechips/integrations-ui/apps/ TCRearViewcamera)**
  - Camera2 Java API from framework Layer
  - CameraManager : frameworks/base/core/java/android/hardware/camera2/CameraManager.java
    - openCamera()
    - createCaptureSession()
    - setRepeatingRequest()
  - From here it is a binder interface to camera service (ICameraService).

- **Framework service (cameraserver PID 320)**
  - The binder endpoint for ICameraService will be implemented in cameraserver.
  - frameworks/av/services/camera/libcameraservice/: (check CameraService.cpp /.h)
  - Key classes/functions
    - connectDevice
    - Connect
    - CameraProviderManager
    - Camera3Device
  - CameraService (cameraserver PID 320) calls HIDL provider legacy/0 through Provider manager

- **Provider manager and deamon**
  - This is where cameraserver connects to HIDL provider (legacy/0):
    - frameworks/av/services/camera/libcameraservice/common/CameraProviderManager.cpp
  - This uses provider deamon to connect to telechip camera HAL:
    - android.hardware.camera.provider@2.4/2.5::ICameraProvider opens legacy telechip HAL module

- **HAL3 code in hardware/telechips/camera/libcamera_v3) runs:**
  - Initialize and configure streams
  - process capture requests

- **HAL talks to driver (V4L2/ISP/etc.)**

# Call flow from Camera / RVC app to HAL

- **# Where CameraManager class is**
- grep -R "class CameraManager" -n frameworks/base/core/java/android/hardware/camera2/CameraManager.java

- **# Find openCamera implementation references**
- grep -R "openCamera(" -n frameworks/base/core/java/android/hardware/camera2 | head

- **# Find ICameraService AIDL**
- ls frameworks/base/core/java/android/hardware/ICameraService.aidl

- **# Find where native CameraService implements connect/open**
- grep -R "class CameraService" -n frameworks/av/services/camera/libcameraservice
- grep -R "connectDevice" -n frameworks/av/services/camera/libcameraservice/CameraService.cpp
- grep -R "connect(" -n frameworks/av/services/camera/libcameraservice/CameraService.cpp | head

- **# Find provider manager usage**
- grep -R "CameraProviderManager" -n frameworks/av/services/camera/libcameraservice

- grep -n "openCamera" frameworks/base/core/java/android/hardware/camera2/CameraManager.java | head -n 30

# HIDL

# HIDL

- HAL daemons communicate via Binder messages sent via **/dev/hwbinder** with manager **hwservicemanager**
- Interfaces defined in a C++ style language called **HIDL**
- Tool **hidl-gen** creates proxy/stub classes in C++ or Java
- Most HAL interfaces are defined in hardware/interfaces
  - also some in frameworks/hardware/interfaces
  - and system/hardware/interfaces

# Listing HALs

- HIDL HALs are hardware services, which uses **/dev/hwbinder**
- We can list them using **lshal**

```
maneesh@maneesh:/media/maneesh/e88ea721-fad1-4d7c-86d4-79b39aaac659/Forvia-training-Android14/aosp$ adb devices
List of devices attached
0.0.0.0:6520    device

maneesh@maneesh:/media/maneesh/e88ea721-fad1-4d7c-86d4-79b39aaac659/Forvia-training-Android14/aosp$ adb shell
xrda3car:/ $ lshal
Warning: Skipping "android.frameworks.sensorservice@1.0::ISensorManager/default": cannot be fetched from service manager (null)
Warning: Skipping "android.hardware.audio.effect@7.0::IEffectsFactory/default": cannot be fetched from service manager (null)
Warning: Skipping "android.hardware.audio@7.0::IDevicesFactory/default": cannot be fetched from service manager (null)
Warning: Skipping "android.hardware.audio@7.1::IDevicesFactory/default": cannot be fetched from service manager (null)
Warning: Skipping "android.hardware.automotive.evs@1.0::IEvsEnumerator/default": cannot be fetched from service manager (null)
Warning: Skipping "android.hardware.automotive.evs@1.1::IEvsEnumerator/default": cannot be fetched from service manager (null)
Warning: Skipping "android.hardware.media.c2@1.0::IComponentStore/software": no information for PID 524, are you root?
Warning: Skipping "android.hidl.allocator@1.0::IAllocator/ashmem": no information for PID 374, are you root?
Warning: Skipping "android.hidl.base@1.0::IBase/ashmem": cannot be fetched from service manager (null)
Warning: Skipping "android.hidl.base@1.0::IBase/default": cannot be fetched from service manager (null)
Warning: Skipping "android.hidl.base@1.0::IBase/software": cannot be fetched from service manager (null)
Warning: Skipping "android.hidl.manager@1.0::IServiceManager/default": no information for PID 148, are you root?
| All HIDL binderized services (registered with hwservicemanager)
VINTF R Interface                                          Thread Use Server Clients
FM    ? android.frameworks.sensorservice@1.0::ISensorManager/default N/A        N/A
DM,FC ? android.hardware.audio.effect@7.0::IEffectsFactory/default   N/A        N/A
DM,FC ? android.hardware.audio@7.0::IDevicesFactory/default          N/A        N/A
DM,FC ? android.hardware.audio@7.1::IDevicesFactory/default          N/A        N/A
FM    ? android.hardware.automotive.evs@1.0::IEvsEnumerator/default  N/A        N/A
FM    ? android.hardware.automotive.evs@1.1::IEvsEnumerator/default  N/A        N/A
FM    Y android.hardware.media.c2@1.0::IComponentStore/software      N/A        524
FM    Y android.hardware.media.c2@1.1::IComponentStore/software      N/A        524
FM    Y android.hardware.media.c2@1.2::IComponentStore/software      N/A        524
FM    Y android.hidl.allocator@1.0::IAllocator/ashmem                N/A        374
X     ? android.hidl.base@1.0::IBase/ashmem                          N/A        N/A
X     ? android.hidl.base@1.0::IBase/default                         N/A        N/A
X     ? android.hidl.base@1.0::IBase/software                        N/A        N/A
DC,FM Y android.hidl.manager@1.0::IServiceManager/default            N/A        148
FM    Y android.hidl.manager@1.1::IServiceManager/default            N/A        148
FM    Y android.hidl.manager@1.2::IServiceManager/default            N/A        148
FM    Y android.hidl.token@1.0::ITokenManager/default                N/A        148

| All HIDL interfaces getService() has ever returned as a passthrough interface;
| PIDs / processes shown below might be inaccurate because the process
| might have relinquished the interface or might have died.
| The Server / Server CMD column can be ignored.
| The Clients / Clients CMD column shows all process that have ever dlopen'ed
```

# HIDL example

hardware/interfaces/light/2.0/ILight.hal

```
package android.hardware.light@2.0;

interface ILight {
    setLight(Type type, LightState state) generates (Status status);
    getSupportedTypes() generates (vec<Type> types);
};
```

- Version **@2.0**: allows backwards compatibility - framework will choose the latest version of HAL supported by vendor
- Return value defined by **generates**
- Variables are typed (e.g. Status, Type) – will check this in next slide

# HIDL example

hardware/interfaces/light/2.0/ILight.hal

```
package android.hardware.light@2.0;

enum Status : int32_t {
    SUCCESS,
    LIGHT_NOT_SUPPORTED,
    BRIGHTNESS_NOT_SUPPORTED,
    UNKNOWN,
};
enum Type : int32_t {
    BACKLIGHT,
    KEYBOARD,
    BUTTONS,
    BATTERY,
    NOTIFICATIONS,
    ATTENTION,
    BLUETOOTH,
    WIFI,
    COUNT,
};
[...];
```

# hidl-gen

- **hidl-gen** generates code from the HAL file
- Example:

```
$ hidl-gen -L c++-impl -o device/sirius/marvin/light \
-r vendor.example:vendor/sirius android.hardware.light@2.0
$ tree device/sirius/marvin
device/sirius/marvin
`-- light
    |-- Light.cpp
    `-- Light.h
```

# The template HAL code

Light.cpp

```
#include "Light.h"

namespace android::hardware::light::implementation {

    Return<::android::hardware::light::V2_0::Status> Light::setLight(::android::hardware::ligh
t::V2_0::Type type, const ::android::hardware::light::V2_0::LightState& state) {
        // TODO implement
        return ::android::hardware::light::V2_0::Status {};
    }

    Return<void> Light::getSupportedTypes(getSupportedTypes_cb _hidl_cb){
        // TODO implement
        return Void();
    }
}
```

# Callbacks

- Simple types are returned by the function
- Strings, arrays and structures are returned as callbacks using `_hidl_cb`:
- Example

```
Return<void> Light::getSupportedTypes(getSupportedTypes_cb _hidl_cb){
    std::vector<V2_0::Type> types;
    types.push_back(V2_0::Type::BACKLIGHT);
    _hidl_cb(types);

    return Void();
}
```

# Implementing the daemon

- Code to set the HAL daemon (native service) running

```
#include "Light.h"

int main() {
    sp<ILight> service = new Light();

    configureRpcThreadpool(1, true);
    status_t status = service->registerAsService();
    if (status == OK) {
        ALOGI("Light HAL Ready");
        joinRpcThreadpool();
    }

    ALOGE("Cannot register Light HAL service");
    return 1;
}
```

# Instances

- Sometimes you need several instances of the same HAL
  - Example: IBroadcastRadio has an instance for each broadcast statndard
- Give the instance name as a parameter to `registerAsService`
  - Default name is "default"

This shows two instances of IBroadcastRadio, "amfm" and "dab":

```
    status_t status = service->registerAsService("dab");
```

```
$ lshal | grep IBroadcastRadio
DM,FC Y android.hardware.broadcastradio@2.0::IBroadcastRadio/amfm   0/2   456   678 248
DM,FC Y android.hardware.broadcastradio@2.0::IBroadcastRadio/dab    0/2   456   678 248
```

# Running the HAL daemon

- Install the daemon into `/vendor/bin/hw` or `/system/bin/hw`
- Install the rc init script into `/vendor/etc/init` or `/system/etc/init`

In the case of the lights HAL, this rc file would be needed:

`light-service.rc`

```
service light-hal-2-0 /vendor/bin/hw/android.hardware.light@2.0-service.example
    interface android.hardware.light@2.0::ILight default
    class hal
    user system
    group system
```

# Building - Android.bp

```
cc_binary {
    name: "android.hardware.light@2.0-service.example",
    vendor: true,
    relative_install_path: "hw",
    srcs: [
        "Light.cpp",
        "service.cpp",
    ],
    shared_libs: [
        "libhidlbase",
        "libutils",
        "android.hardware.light@2.0",
        "liblog",
    ],
    init_rc: ["light-service.rc"],
    vintf_fragments: ["light-service.xml"],
}
```

Must set `proprietary: true` or `vendor: true`

Setting `relative_install_path: "hw"` installs into `/vendor/bin/hw`

# Practice :- HIDL HAL Demo 1

# Practice :- HIDL HAL Demo 2

# V2X (Vehicle to Everything)

# What is V2X

- Vehicle-to-Everything (V2X) technology enables cars to communicate with their surroundings and makes driving safer and more efficient for everyone.
- By making the invisible visible, V2X warns the driver of road hazards, helping reduce traffic injuries and fatalities.
- In addition to improving safety, V2X helps to optimize traffic flow, reduce traffic congestion and lessen the environmental impact of transportation.
- V2X enables vehicles to communicate with:
    - Other vehicles (V2V)
    - Infrastructure like traffic signals (V2I)
    - Pedestrians and cyclists (V2P)
    - Network/cloud services (V2N)
- Makes the “invisible visible” by sharing real-time information
- Acts as a foundation for connected and autonomous mobility

# Why V2X is Needed

- Human vision and sensors have limited range
  - Drivers often cannot see:
  - Hidden road hazards
  - Sudden braking vehicles
  - Pedestrians behind obstacles
- V2X extends vehicle awareness beyond line-of-sight
- Complements cameras, radar, and LiDAR systems
- Acts as a foundation for connected and autonomous mobility

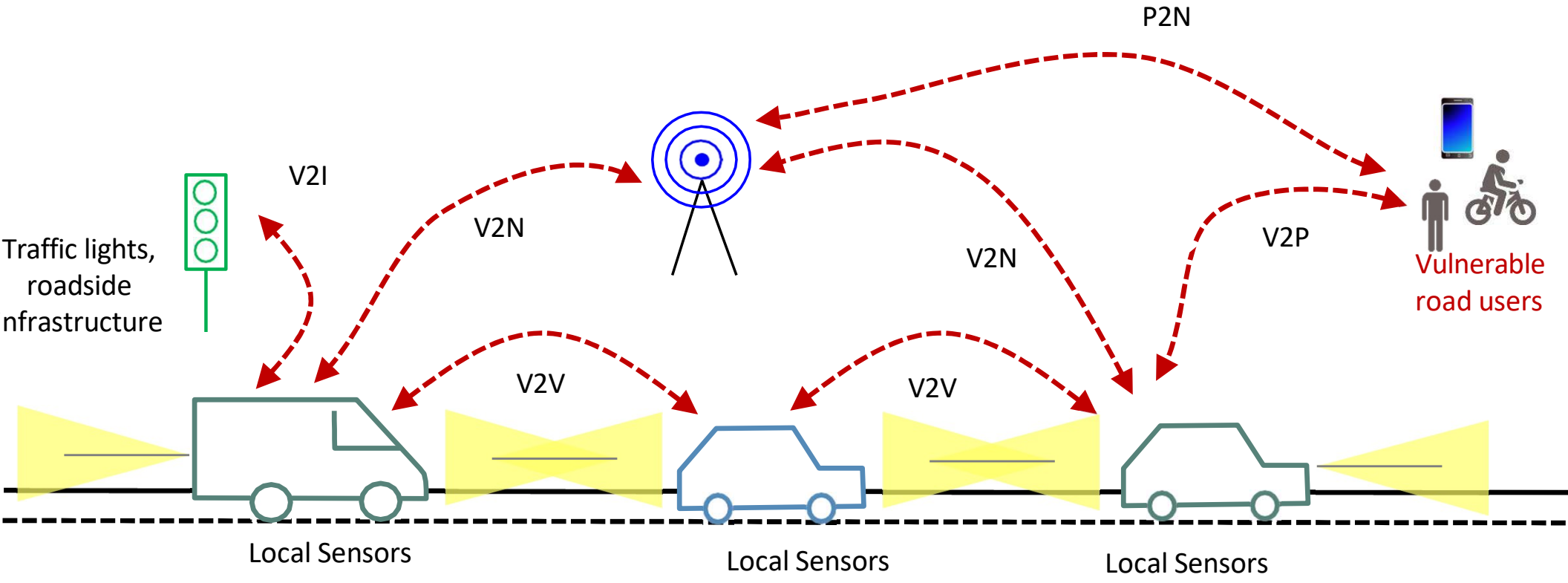
# Safety Benefits of V2X

- Early warnings for:
  - Collision risks
  - Emergency vehicle approach
  - Blind-spot hazards
  - Road construction and accidents
- Helps reduce traffic injuries and fatalities
- Enables faster driver reaction time
- Critical for ADAS and autonomous driving systems

# Basic V2X Concept

Together, these form part of what we describe as Cooperative Intelligent Transport Systems (C-ITS).

V2X – Vehicle to Everything
V2I – Vehicle to Infrastructure
V2P – Vehicle to Pedestrian
V2V – Vehicle to Vehicle
V2N – Vehicle to Network
P2N – Pedestrian to Network



# Google Automotive Services (GAS)

# What Is Google Automotive Services (GAS)?

- Google Automotive Services (GAS) is a suite of Google apps and services for Android Automotive OS
- Designed specifically for in-vehicle infotainment (IVI) systems
- Runs natively on the car head unit (not projection like Android Auto)
- Enables a Google-powered in-car experience
- Key idea: Android Automotive + Google services = GAS

# What Is Core Components of GAS ?

- GAS includes the following Google-certified apps:

- **Google Maps**
  - Turn-by-turn navigation
  - Real-time traffic updates
  - EV routing and charging info

- **Google Assistant**
  - Voice control for navigation, media, calls
  - Hands-free driving experience

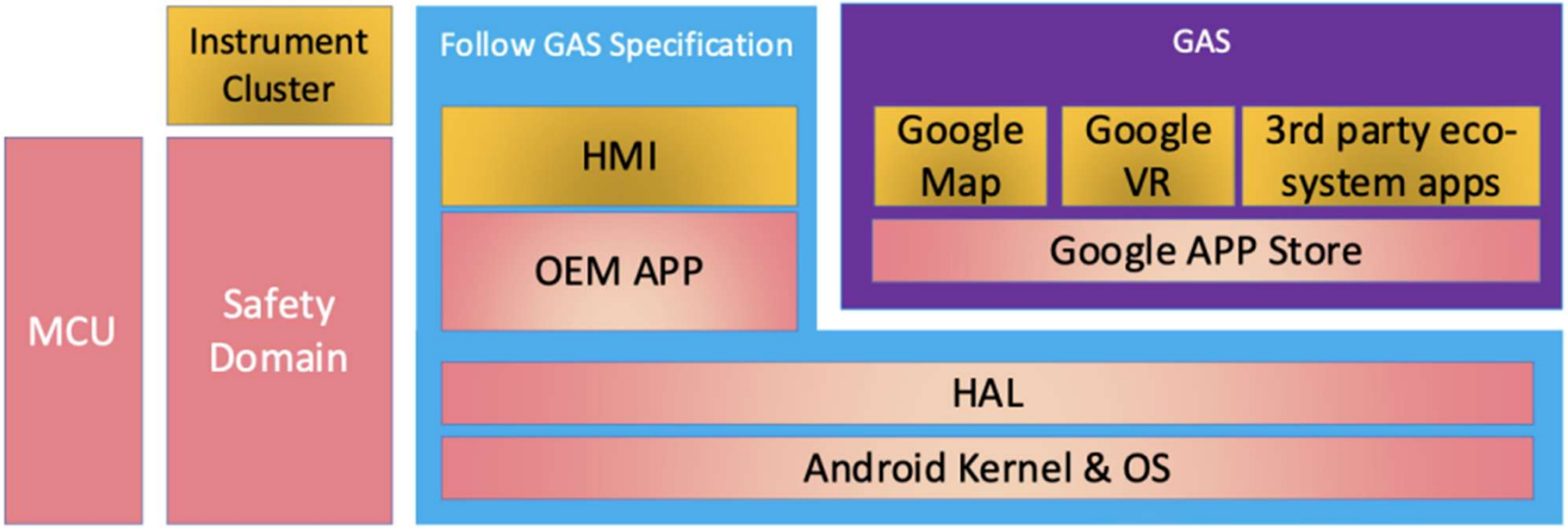
- **Google Play Store**
  - Automotive-optimized apps
  - Media, navigation, parking, EV apps

# GAS vs Android Auto

| Feature | Android Auto | Android Automotive + GAS |
|---|---|---|
| Runs on | Phone | Vehicle head unit |
| Internet | Phone-based | Vehicle modem |
| UI Control | Phone-driven | OEM-customizable |
| Offline Support | Limited | Better |
| Certification | AA | GAS + CTS |

NOTE : - GAS is **embedded in Vehicle head-Unit but** , Android Auto is **projection-based**

# GAS Architecture Overview



# GAS Certification & Requirements

- OEM must:
  - Sign GAS licensing agreement with Google
  - Pass CTS + GTS + VTS
  - Meet UX and safety guidelines
- Hardware requirements:
  - Sufficient CPU/GPU/RAM
  - Automotive-grade SoC
- Internet connectivity required (LTE/5G/Wi-Fi)

# Future trend in Automotive

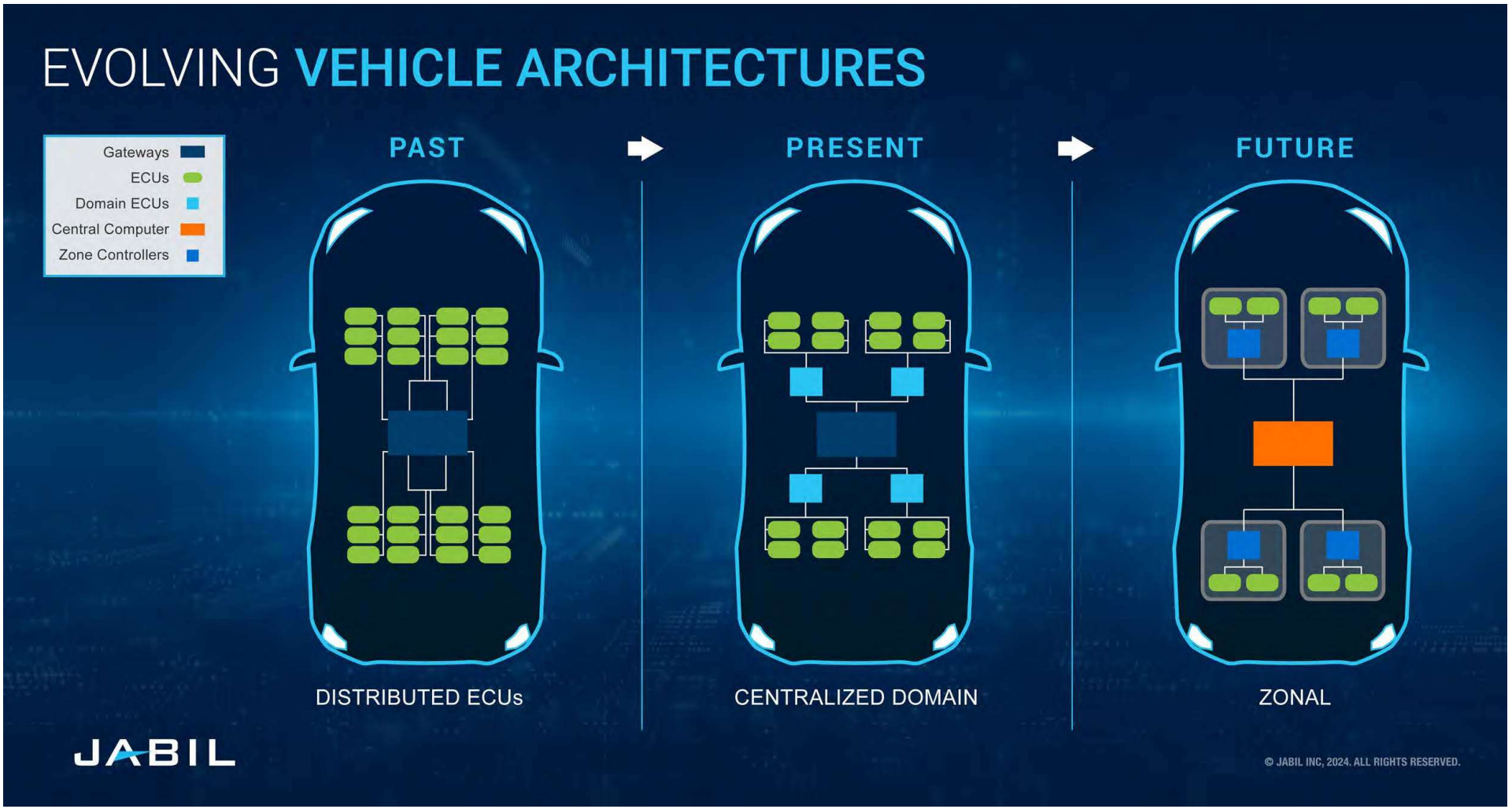
# Software-Defined Vehicle?

- "Software-defined vehicle" is a term for conceptualizing the industry's shift from a static, electromechanical-focused product to an upgradeable, platform-based mobility solution in which the architecture of the vehicle is defined by the software, not the hardware.
- Tesla was a first-mover, expanding the limits of traditional automotive software by designing a computer network and then building vehicular firmware around it. The concept is to think of a car, like today's electric vehicles (EVs), as essentially a high-performance computer (HPC) on wheels, with software providing the means for a personalized, two-way "conversation" between vehicles and their manufacturers, as well as enabling connections across the entire ecosystem.
- Here are some examples of the range of services the fully connected car's software can deliver:
  - Highly automated driving functionality,
  - Advanced human-machine interfaces (HMIs), like AI-powered virtual assistants,
  - Enhanced driver services, like smart access, virtual reality heads-up displays, theft prevention and vehicle tracking, emergency assistance, navigation, and other travel intelligence,
  - Personalized infotainment that moves with you,
  - Customer engagement insights that drive improvement back up the value chain,
  - Telematics for driving prediction services and additional optimization,
  - Over the air updates (OTA) and quality fixes.

# V2X (Vehicle to Everything)

- V2X makes it possible for vehicles to communicate with each other — as well as other connected elements comprising the infrastructure of "smart cities," such as smart traffic lights, railroad crossings, etc.
- A fully developed V2X future will enable, essentially, all actors on or near a road to communicate their position and trajectory back and forth, in near real-time, via a mix of complementary communications services, including cellular, short-range radio technology, broadcast systems as well as additional networking elements, like unmanned-aerial vehicle (UAV) and satellites.

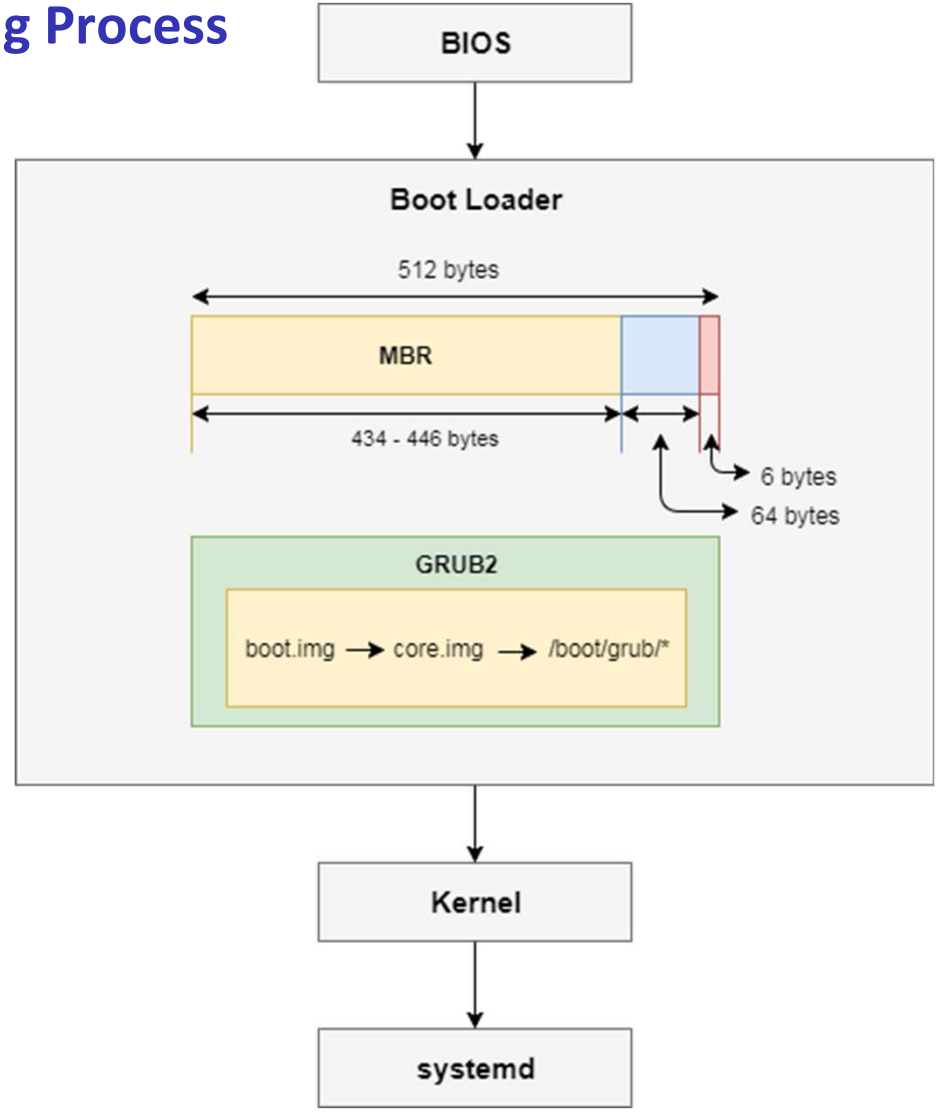
# Vehicle Architecture



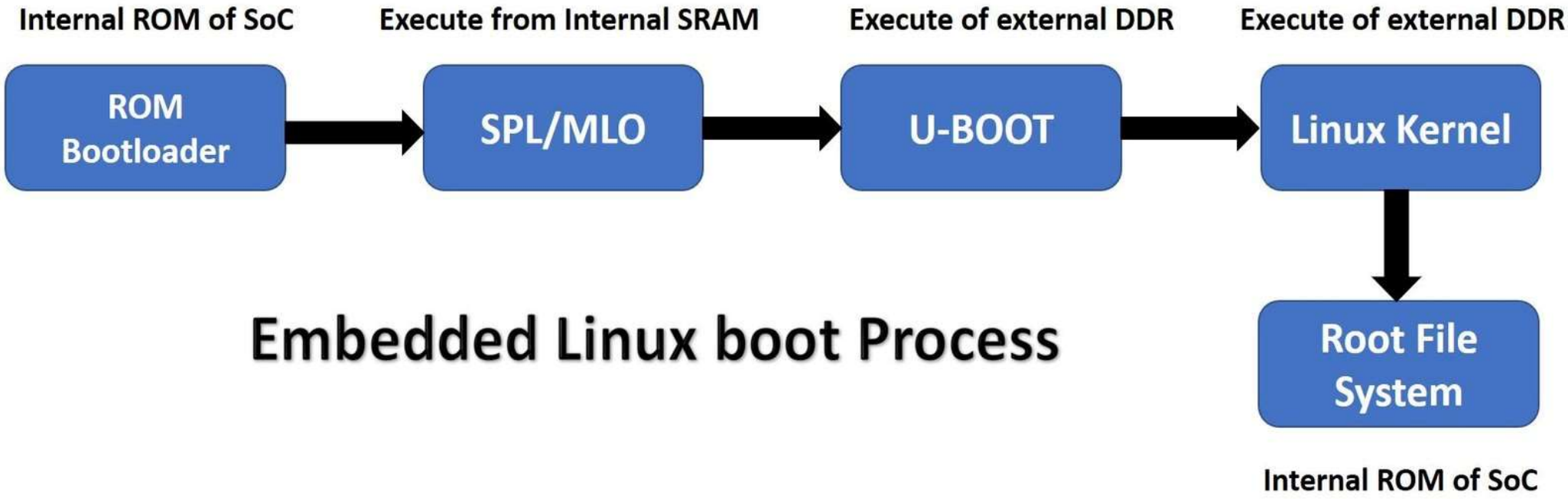
# Android Boot time Optimisations

# Android Boot time Optimisations

# Linux PC / Ubuntu Booting Process

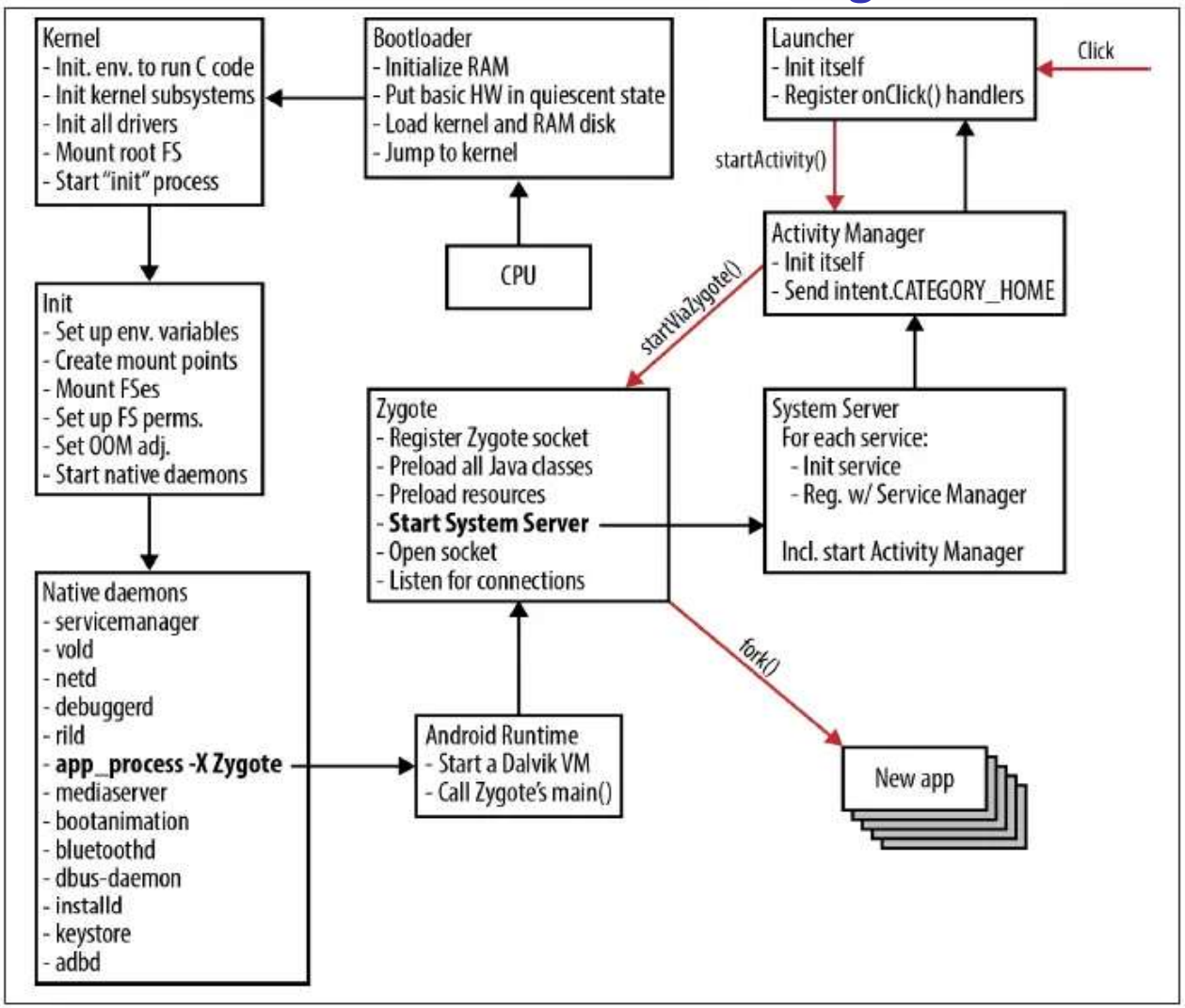


## Embedded Linux Booting Process




Embedded Linux boot Process

# Android Booting Process




**Android Boot Flow (High Level)**

1. **Boot ROM** (SoC internal)
2. **BL1 / BL2** (Telechips bootloader)
3. **U-Boot**
4. **Linux Kernel**
5. **Init (init.rc)**
6. **Zygote**
7. **System Server**
8. **Launcher / HMI**

→ Boot optimization means reducing time at *each stage*

# Telechip Boot Time Categories

**Boot Time Categories**

- **Cold Boot** – Power OFF → ON
- **Warm Boot** – Reboot
- **Fast Cold Boot** – Optimized cold boot (Telechips feature)

**Focus of this training:**

- Fast Cold Boot on TCC805x Android 10

# Telechips Fast Cold Boot – Overview

**Telechips SDK provides Fast Cold Boot by:**

- Kernel driver modularization
- Reducing kernel console overhead
- Faster Android framework startup
- Removing unnecessary system components
- Optimized for Telechips EVB (AOSP Car Product)

# Kernel-Level Optimization

**Problem:**
- Monolithic kernel loads many drivers at boot
- Increases kernel init time

**Solution:**
- Convert heavy drivers into **Loadable Kernel Modules (LKM)**
- Means we can load the driver at the run-time

**Telechips Modularized Drivers Examples:**
- SDMMC
- Sound
- USB

# Kernel-Level Optimization

**Problem:**
- Monolithic kernel loads many drivers at boot
- Increases kernel init time

**Solution:**
- Convert heavy drivers into **Loadable Kernel Modules (LKM)**
- Means we can load the driver at the run-time

**Telechips Modularized Drivers Examples:**
- SDMMC
- Sound
- USB

**Module Loading Strategy –**

**Two Options:**
- Load during boot (init.rc)
- Load on-demand (later stage after boot)

**Benefit:**
- Kernel boots faster
- Non-critical drivers delayed

# Kernel-Level Optimization

**Disable Kernel Console Output**

**Problem:**
- Excessive printk() logs slow down boot

**Solution:**
- Enable `quiet` boot option

**Change in BoardConfig.mk:**

```
BOARD_KERNEL_CMDLINE := \
console=ttyAMA0,115200n8 androidboot.console=ttyAMA0 \
androidboot.hardware=$(TARGET_BOARD_PLATFORM) \
androidboot.selinux=permissive printk.devkmsg=on quiet
```

# Android Framework Optimization

**Key Areas:**

- Boot animation
- Zygote startup
- System applications

Focus is on **reducing workload before launcher starts**

## Boot Animation Optimization

**Why important?**
- Boot animation blocks perceived boot completion

**Optimization Tips:**
- Reduce animation resolution
- Reduce frame count
- Disable animation for demo systems

## Zygote Optimization

**Zygote Role:**
- Preloads framework classes
- Forks app processes

**Optimization Techniques:**
- Reduce preloaded classes
- Avoid unnecessary services at boot
- Faster Zygote → Faster System Server

# Remove Unnecessary System Apps

**Default Apps to Remove (Telechips SDK):**

- Calendar
- Contacts
- DeskClock
- Email
- ExactCalculator
- Camera2
- MusicFX
- NfcNci
- EmergencyInfo

**Result:**

- Reduced boot services
- Lower RAM usage

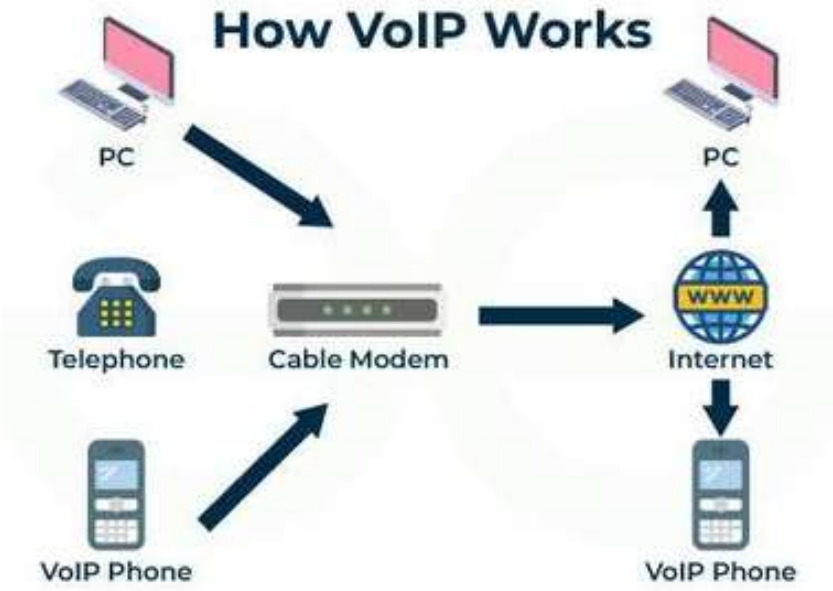
**Where to Remove System Apps**

- device/telechips/car_tccxxxx/device.mk
- product packages list

Or disable via:
PRODUCT_PACKAGES -= Calendar

# Practice -> Audio flow in AOSP

# VoIP




**What is VoIP?**

Voice over Internet Protocol (VoIP) is a technology that allows voice communication over the internet. By converting voice signals into digital data packets, VoIP enables users to make calls from **computers, smartphones, or VoIP phones**.

**How Does Voice Over Internet Protocol (VoIP) Works?**

- Voice is converted into a digital signal by VoIP services that travel over the Internet.
- If the regular phone number is called, the signal is converted to a regular telephone signal i.e. an analog signal before it reaches the destination.
- VoIP allow to make a call directly from a computer having a special VoIP phone or a traditional phone connected to a special adapter.
- Wireless hot spots in locations such as airports, hospitals, cafes, etc allow you to connect to the Internet and can enable you to use VoIP service wirelessly.

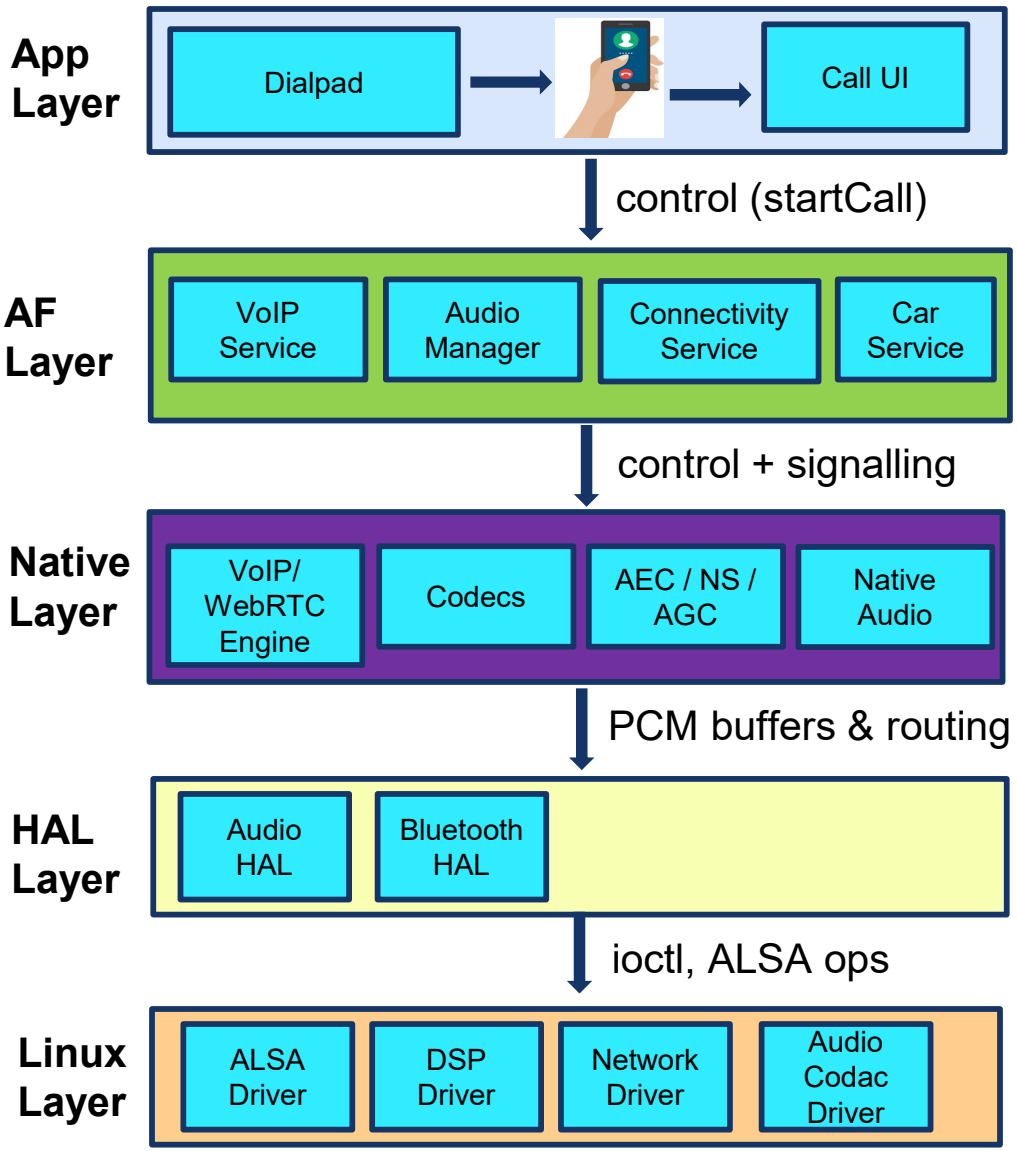
# AOSP VoIP Architecture

AOSP **does NOT** provide a complete VoIP solution, but provides:

- Networking APIs (WebSocket, TCP/UDP)
- Media APIs (AudioRecord/AudioTrack)
- SIP API (deprecated, but still present)
- WebRTC support (integrated externally)
- Audio routing + policy
- Permissions & platform integration

So In Android VoIP is =>

- **Application +**
- **Native Media Engine +**
- **Android Audio Path.**



**VoIP UI**
- Dialer UI
- Incoming call screen
- Mute, Hold, End
- Settings

**AudioManager**
- setMode (MODE_IN_COMMUNICATION)
- requestAudioFocus
- setSpeakerphoneOn
- setBluetoothScoOn

**Connectivity Framework**
- sockets
- WebSockets
- HTTP/TLS
- SIP (deprecated)

**Car Framework (AAOS)**
- CarService
- VHAL → Steering button events

**WebRTC Engine**:
- Encode PCM → RTP
- Decode RTP → PCM
- Signaling (SDP/SIP)
- AEC/NS/AGC

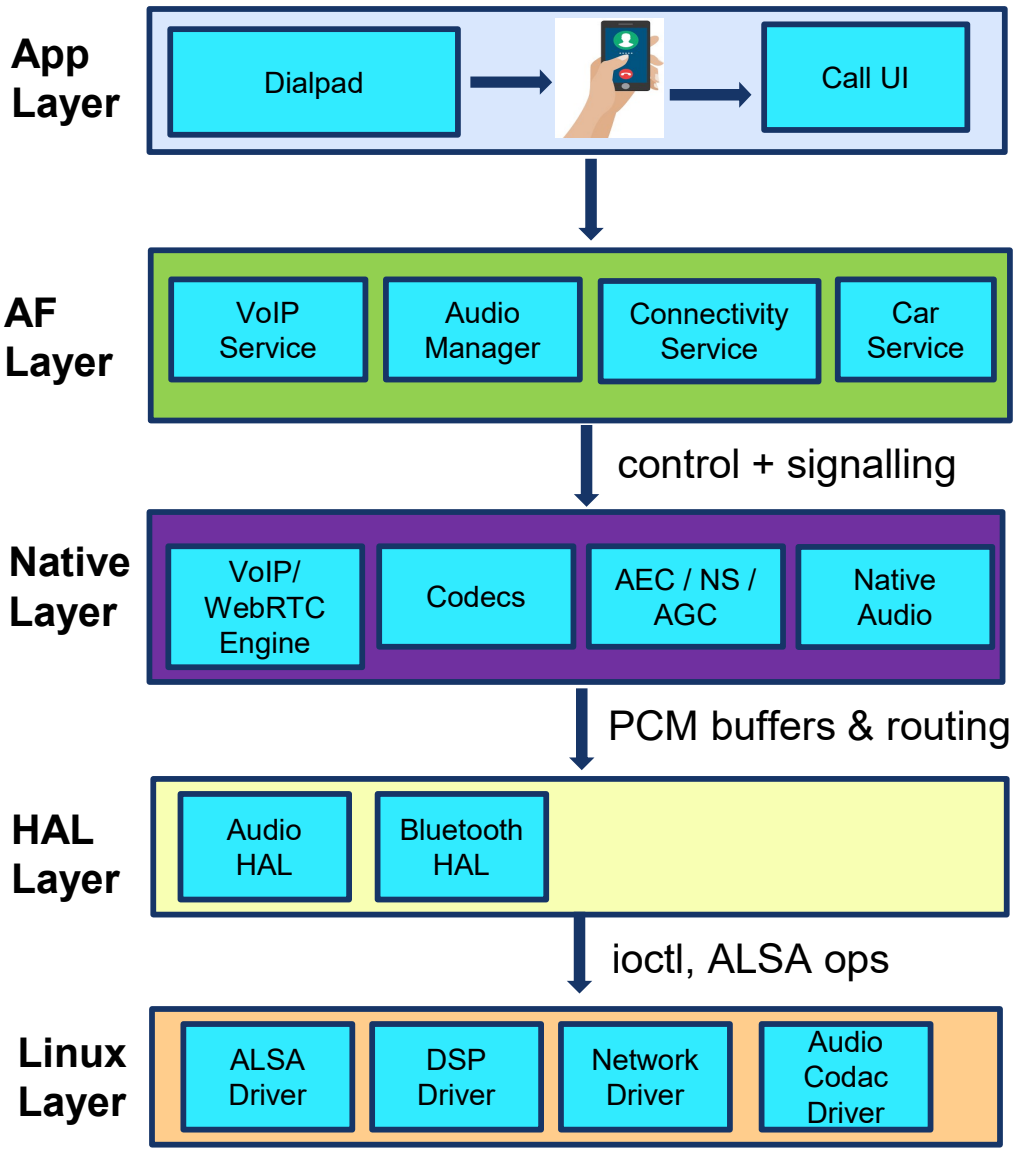
**Codecs**
- Opus
- G.711
- AAC
- AMR-NB/WB
- iLBC

**AEC / NS / AGC**
- WebRTC APM (Audio Processing Module)
- Qcomm/Telechip DSP (AEC/NS/AGC)

**Native Audio I/O**
- OpenSL ES
- AAudio
- libaudioclient (Mic PCM -> AudioRecord / Spek PCM -> AudioTrack etc.)

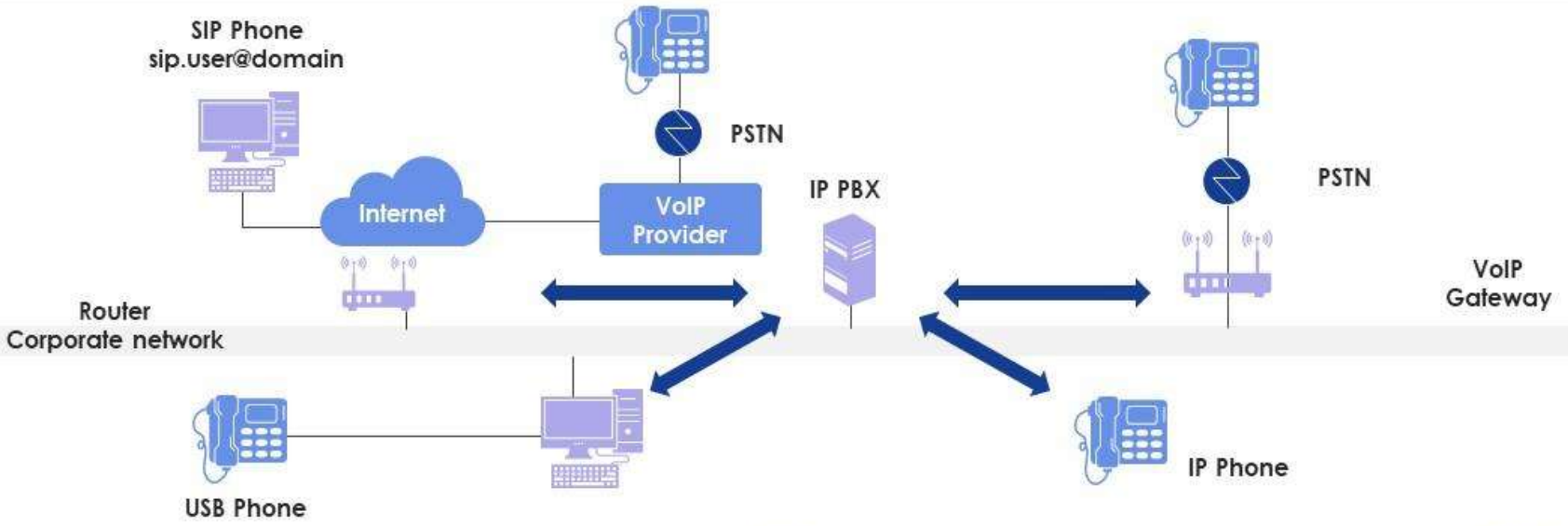
**Audio and Bluetooth HAL**
- mic path
- speaker path
- BT SCO path
- DSP AEC routing
- Codac gain control etc..

**This is where VoIP audio is routed to real hardware.**



# Voice Over Internet Protocol (VoIP) architecture

The purpose of the mentioned slide is to highlight the elements of voice over network protocol (VoIP) application architecture. It includes elements such as SIP phone, router, PSTN, VoIP gateway, etc.




Description

01 **SIP Phone**: IP telephony leverages the foundation provided by VoIP, extending

02 **Router**: Its functionalities with various software-based features that operate on computers

03 **PSTN**: This expansion aims to replace the current Public Switched Telephone Network (PSTN) infrastructure

04 **VoIP Gateway**: Establish connections through data networks rather than traditional telephone networks

This slide is 100% editable. Adapt it to your needs and capture your audience's attention.

# Telechip IPC Comm

# Telechips IPC Overview

- IPC allows A72, A53, and MCU to exchange data.
- Used for -
  - vehicle signals
  - Diagnostics
  - subsystem control
- Fast, low-latency communication using mailbox hardware.

# Telechips IPC Architecture

**IPC Driver (Linux char device)**

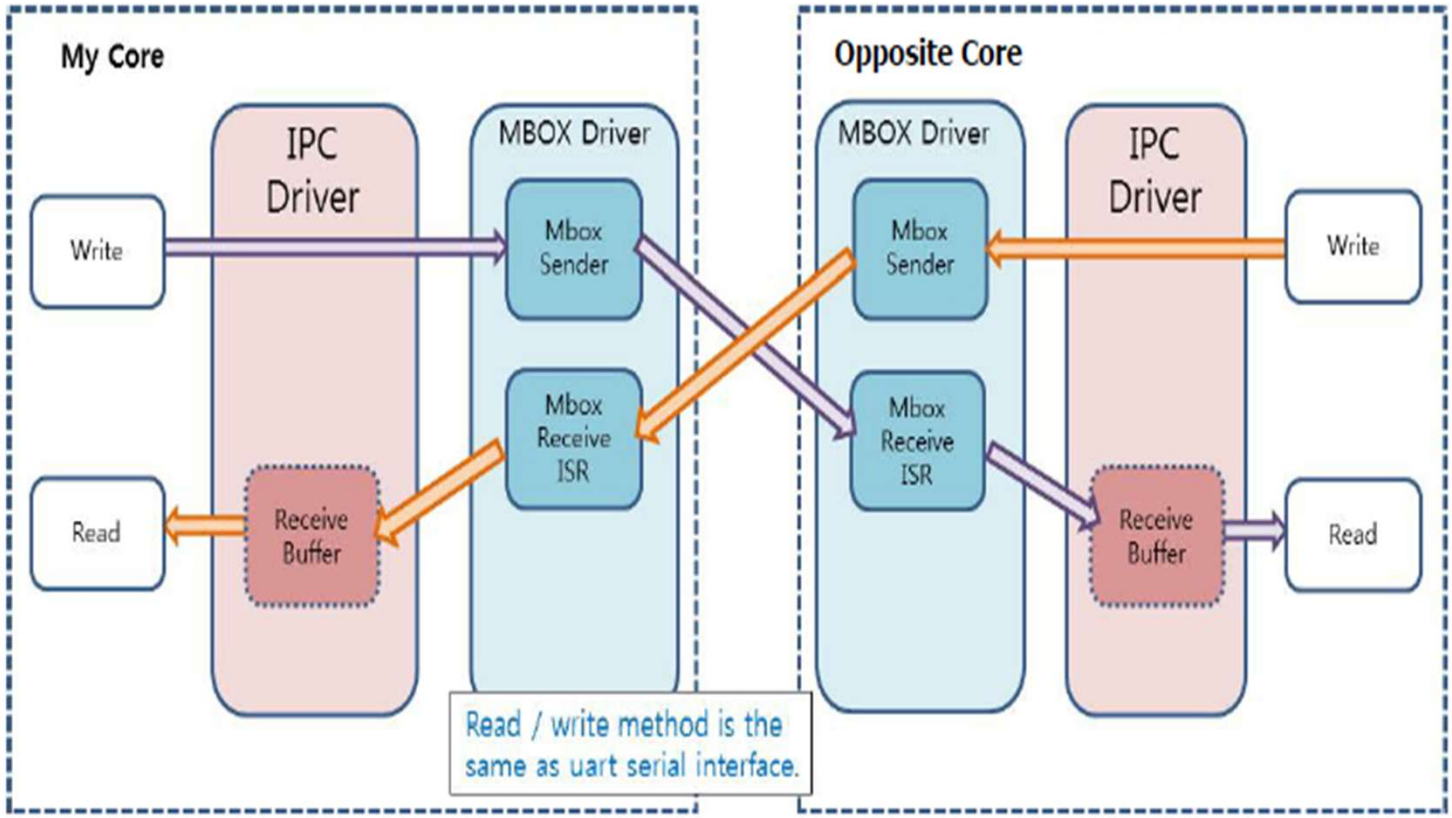
- Exposes: /dev/tcc_ipc_ap, /dev/tcc_ipc_micom
- Handles buffering, IOCTL, blocking reads

**Mailbox (MBOX) Driver**

- Manages mailbox registers
- Handles interrupts between cores

**Mailbox Hardware (MBOX HW)**

- Dedicated hardware peripheral
- Generates doorbell IRQs
- Transfers message headers/ACK signals




**https://developer.arm.com/documentation/ddi0306/b/CHDGHAIG**

# Mailbox Hardware

- Dedicated hardware peripheral in Telechips SoC.
- Provides channels for TX/RX.
- Generates interrupts to opposite core.

# Why Mailbox?

- Very fast
- Non-blocking
- Works across heterogeneous cores
- No need for shared memory polling

# IPC Driver Read/Write Flow

- TX Buffer: 512 bytes max.
- RX Buffer: 512 KB.
- IOCTL: readiness, ping test, timeout, flush.
- Behavior similar to UART but interrupt-driven.

# Practice :- Telechip IPC Demo

# THANK YOU